Project Planning and Management

1. Project Proposal

1.1 Overview

The project builds an end-to-end real-time data pipeline for IoT sensor data. A device simulator publishes readings (temperature, humidity, CO2, power draw) from a fleet of devices spread over several sites. The readings travel through a message broker, are validated and enriched by a streaming job, are stored in layered storage, and are served to a live dashboard. Bad records are quarantined instead of being lost or crashing the pipeline.

The whole system runs from one Docker Compose command, uses free tools only, and is tested with late, duplicate and malformed events.

1.2 Problem statement

Organizations that monitor equipment and buildings receive thousands of sensor readings per second. Batch processing shows problems hours late, raw data contains duplicates, gaps and corrupt values, and dashboards built on unvalidated data cannot be trusted. The project shows how to deliver validated, fresh sensor data to analysts within seconds, with measurable quality and reliability.

1.3 Objectives

- O1. Ingest simulated sensor events from at least 500 devices at a sustained rate of 500 events per second or more.
- O2. Process events in a streaming job that parses, validates, deduplicates, handles late data with watermarks, and computes windowed aggregates.
- O3. Route invalid events to a dead-letter store with the failure reason, so nothing is silently dropped, and check data quality with Great Expectations suites.
- O4. Store data in layers (raw, clean, aggregated) with Apache Parquet files for the lake, and expose a relational serving layer in SQL Server Developer for analytics.
- O5. Show live device and site metrics on a dashboard with data no more than 15 seconds old.
- O6. Meet the KPI targets in section 5 and prove it with a measured test report.
- O7. Deliver everything the guidelines require: documentation, GitHub repository, automated tests, one-command setup, presentation and infographic.
- O8. Orchestrate maintenance and reporting with Airflow DAGs: stream_health_check, dq_daily_report, lake_compaction, gold_backfill and warehouse_maintenance.

1.4 Scope Management

In scope

- A Python device simulator with configurable rate, device count and fault injection (late, duplicate, malformed, out-of-range events).
- Kafka topics for raw events, with a dead-letter topic.
- PySpark Structured Streaming jobs for validation, deduplication, enrichment and windowed aggregation.
- Layered storage: raw and clean Apache Parquet files, plus a SQL Server Developer serving database.
- A dashboard with live readings, aggregates, device health and data-quality panels.
- Data quality checks with Great Expectations, pipeline monitoring metrics and pytest automated tests.
- Docker Compose deployment, GitHub repository with README, and all documents from the guidelines.
- Airflow orchestration: five DAGs for stream health, daily data-quality reporting, lake compaction, gold-layer backfill and warehouse maintenance.

Out of scope

- Real physical devices or hardware firmware (data is simulated).
- Paid or cloud-only services, including paid Azure streaming services.
- Machine learning or predictive maintenance models (so the infographic has no AI components section).
- Multi-node production clusters, Kubernetes, and high-availability failover.
- Full user authentication system and mobile apps.
- Alert delivery to external channels (email, SMS). Threshold alerts appear on the dashboard only.

1.5 Proposed technology stack

Layer	Tool	Role
Data source	Python simulator	Generates sensor events with fault injection
Messaging	Apache Kafka	Durable, partitioned event log; raw topic keeps 7 days, plus a dead-letter topic
Stream processing	PySpark Structured Streaming	Per-event validation, deduplication, watermarks, windows

Layer	Tool	Role
Lake storage	Apache Parquet files	Raw and clean layers for history and replay
Serving layer	SQL Server Developer	Relational tables for the dashboard and analysts
Data quality	Great Expectations	Expectation suites on the stored clean Parquet and SQL Server tables, validation results, daily quality report
Orchestration	Apache Airflow	Five DAGs: stream_health_check, dq_daily_report, lake_compaction, gold_backfill, warehouse_maintenance
Dashboard	Grafana	Live visualization on the SQL Server serving tables
Testing	pytest	Unit, integration and quality tests
DevOps	Docker Compose (local), GitHub	One-command run, version control

1.6 Expected deliverables

Stage	Deliverables	Due
1. Planning and management	Proposal, plan, roles, risks, KPIs	19 Oct 2026
2. Literature review	Related work and tool comparison, lecturer sections	19 Oct 2026
3. Requirements gathering	Stakeholders, user stories, functional and non-functional requirements	19 Oct 2026
4. System analysis and design	Diagrams, schema, wireframes	6 Nov 2026
5. Implementation	Source code, repository, README, Docker Compose	30 Nov 2026
6 and 7. Testing, final presentation and reports	Test plan and report, technical documentation, user manual, presentation, infographic, optional demo video	4 Dec 2026

1.7 Assumptions and constraints

- Free-to-use tools only, no budget.

- Low resources.
- The pipeline runs on team laptops with Docker, so throughput targets are sized for a laptop.
- All sensor data is simulated, so no personal or sensitive data is involved.
- Everything is uploaded to GitHub.

2. Project Plan

2.1 Milestones

Date	Milestone	Exit criteria
19 Oct 2026	Planning, literature review and requirements submitted	All three documents on GitHub
6 Nov 2026	System analysis and design submitted	All required diagrams, schema and wireframes reviewed
23 Nov 2026	Feature freeze (internal)	End-to-end flow runs; only fixes after this date
30 Nov 2026	Implementation submitted	Code, README and one-command Docker Compose setup on GitHub
4 Dec 2026	Testing, final presentation and reports submitted	Test report, technical docs, manual, presentation, infographic

2.2 Gantt chart

Each block marks a week in which the activity is active. Build activities end by 22 Nov, before the 23 Nov feature freeze.

Activity	19 Oct	17-23 Oct	24-30 Oct	31 Oct-6 Nov	7-13 Nov	14-20 Nov	21-30 Nov	1-4 Dec
Planning, literature review, requirements	■							
System analysis and design		■	■	■				
Event contract and interface agreement		■						
Repository, branching, Docker Compose skeleton		■						
Test plan (draft in design, final in testing)				■			■	
Simulator, Kafka producers and topics					■	■		
Spark streaming jobs						■	■	

Activity	19 Oct	17-23 Oct	24-30 Oct	31 Oct-6 Nov	7-13 Nov	14-20 Nov	21-30 Nov	1-4 Dec
Great Expectations suites						■	■	
Storage layers (Parquet and SQL Server)						■	■	
Airflow setup and DAGs						■	■	
Dashboard (Grafana)						■	■	
Automated tests and optional CI						■	■	
README and one-command setup						■	■	
Integration, KPI benchmark, bug fixing							■	
Test execution and test report							■	■
Technical documentation and user manual							■	■
Presentation, infographic, optional demo video								■

2.3 GitHub repository

All project files are uploaded to GitHub

Folder	Contents
docs/planning	Planning document and literature review
docs/requirements	Requirements document
docs/design	All diagrams, schema, wireframes
src	Producers, streaming jobs, serving layer, dashboard, Airflow DAGs, Great Expectations suites
tests	Automated tests and data quality checks
docs/testing	Test plan, test report, bug reports
docs/final	Technical documentation, user manual, presentation, infographic, demo video or link
Repository root	README.md (installation, configuration, execution), docker-compose.yml, .env.example without credentials, .gitignore

3. Task Assignment and Roles

Member	Role	Responsibilities	Main deliverables	Backup
Member 1	Team lead, documentation, presentation	Planning, coordination, submissions, README draft, final review	Proposal, Gantt plan, roles table, problem statement, presentation, infographic	Member 4
Member 2	Kafka and simulator	Device simulator, Kafka topics and producers, related-work note	Simulator, topics, sequence and activity diagrams, literature review, optional demo video	Member 3
Member 3	PySpark and data quality	Streaming jobs, validation, deduplication, windows, Great Expectations suites, KPI measurement	Streaming code, DFD, architecture (with Member 2), KPI definitions	Member 2
Member 4	Storage, ER and dashboard	Requirements, storage layers, SQL Server schema, dashboard	Requirements document, use case diagram, ER diagram and schema, wireframes, dashboard, technical documentation, user manual	Member 1
Member 5	QA, DevOps and GitHub	Repository, Docker Compose, Airflow deployment, tests, CI, risk register	Risk register, repo structure, deployment and component diagrams, test plan, test report	Member 3

Working agreements

- One feature branch per task, merged through a pull request.
- Meaningful commit messages and PR descriptions.

- Weekly sync, plus a short check-in at each interface point (simulator to Kafka, Kafka to Spark, Spark to storage, storage to dashboard).
- Interfaces are fixed early: event schema, topic names, table names. Changes go through an issue and a PR.
- Blockers are raised the same day in the team channel.

4. Risk Assessment and Mitigation

Likelihood (L) and impact (I) are rated Low, Medium or High. Member 5 keeps this register and reviews it at each weekly sync.

ID	Risk	L	I	Mitigation	Owner
R1	Data loss between broker, processor and storage	Medium	High	Kafka replication settings and offsets committed after successful write; checkpointing in Spark; raw layer kept for replay; completeness check comparing counts	Member 3
R2	Consumer lag grows under load	Medium	High	Partition topics by device; tune batch interval and shuffle partitions; monitor lag; run load tests before the freeze	Member 3
R3	Schema changes break the pipeline	Medium	Medium	Versioned event schema with explicit fields; schema validation at entry; unknown sensor types and missing required fields routed to the dead-letter queue, extra fields dropped and counted; schema changes only through PR	Member 2
R4	Tool setup problems on different laptops	High	Medium	Docker Compose with pinned image versions; setup guide; early environment test in week 1; shared troubleshooting notes	Member 5
R5	Laptops lack resources for the full stack	Medium	Medium	Resource limits per container; reduced demo profile; scaled-down simulator settings	Member 5
R6	Duplicate or late events corrupt	High	Medium	Deduplicate on event ID with a watermark; event-time windows; late-event tests	Member 3

ID	Risk	L	I	Mitigation	Owner
	aggregates				
R7	Integration problems discovered late	Medium	High	Fixed interfaces in week 1; thin end-to-end slice early; feature freeze on 23 Nov	Member 1
R8	Scope creep (alerting, extra dashboards)	Medium	Medium	Out-of-scope list; extras only after KPIs are met; lead approves additions	Member 1
R9	Team member unavailable	Medium	Medium	Backup owner per role; documented handoffs; small tasks with clear acceptance criteria	Member 1
R10	Missed deadline for a document stage	Low	High	Weekly schedule with owners; draft one week before each due date; lead reviews before submission	Member 1
R11	Dashboard shows stale or wrong data	Medium	Medium	Freshness metric on the dashboard; data quality panel; reconcile dashboard numbers with serving tables	Member 4
R12	Repository mess (large files, secrets, bad history)	Low	Medium	.gitignore, no data dumps or credentials committed, environment files via examples, PR reviews	Member 5
R13	SQL Server Developer container is memory-hungry next to Kafka, Spark and Airflow	High	Medium	Set memory limits per container; test the full stack early on every laptop; keep a reduced demo profile; Developer edition only for development and demo	Member 5
R14	The starting 2-minute watermark	Medium	Medium	Test several lateness thresholds with the simulator before the feature freeze; record the results;	Member 3

ID	Risk	L	I	Mitigation	Owner
	may suit the data badly (unstable aggregates or too much state)			the value changes only through a pull request with test evidence	
R15	Airflow deployment complexity delays the five required DAGs	Medium	Medium	Pinned Airflow image in Docker Compose; set up early; build the simplest DAG first (stream_health_check) and add the others one by one	Member 5
R16	Great Expectations does not fit a streaming flow directly	Medium	Medium	Great Expectations validates the stored clean Parquet and SQL Server tables; fast structural checks stay in Spark; run an early spike to confirm the setup	Member 3
R17	Closed decisions drift (dashboard tool, event contract, ranges, watermark)	Medium	High	Any change goes through an issue and a pull request and is recorded with its date	Member 1

5. KPIs

Targets are initial values. They are validated in the benchmark run in the last implementation week and reported in the test report.

KPI	Definition and measurement boundary	Target	Acceptance test
Processing latency	Time from Kafka ingest time to served time (row queryable in SQL Server). Event-time delay (event time to ingest time) is reported separately, and injected late events are excluded	p95 of 10 s or less; median of 5 s or less	10-minute normal-load run; percentiles computed from stored timestamps

KPI	Definition and measurement boundary	Target	Acceptance test
Throughput	Valid events processed per second by the streaming job; rejected and duplicate events counted separately	500 events/s sustained; 1,000 events/s peak	10-minute sustained run and a peak run; pass if the target rate is met and consumer lag does not keep growing; record hardware and container limits
Data freshness	Current UTC time minus the served time of the newest successfully served event per site, on one host clock. A device that stops sending is shown as silent, not as stale pipeline data	15 s or less	Dashboard panel checked during a normal-load run with timestamped screenshots
Error or dead-letter rate	Share of injected invalid events captured with a reason, and share of valid events lost to pipeline failures after retries	100% of injected invalid events captured; under 0.1% pipeline-caused failures	Fault-injection run; compare injected counts with dead-letter counts
Pipeline uptime (availability)	Share of health-check samples (every 10 s) in which all required services are healthy: Kafka, streaming job, SQL Server, Airflow and dashboard	99% or more over a 2-hour soak	2-hour soak test; recovery time measured separately for each service restart
Recovery time	Time from a service restart until processing resumes with no data loss	60 s or less per tested service	Restart each of Kafka, the streaming job and SQL Server once during a run
Data completeness (extra)	Valid unique events produced by the simulator versus events stored in the clean layer and SQL Server; intentional duplicate suppression is excluded	100%	Reconciliation query after each test run, covering retried writes
Duplicate rate (extra)	Duplicate event IDs in the clean layer and the serving table	0	SQL and Parquet duplicate check

KPI	Definition and measurement boundary	Target	Acceptance test
Consumer lag (extra)	Age of the oldest unprocessed message (now minus its Kafka timestamp), tracked beside lag in messages	5 s or less under normal load	Kafka consumer group metrics during a normal-load run

Measurement plan

- Every record carries four timestamps in UTC: event time, ingest time (Kafka append time), processed time and served time.
- All services run on one host clock, so timestamp differences are comparable.
- Latency, throughput, freshness and lag are computed from these timestamps and shown on a monitoring panel.
- The benchmark runs three scenarios: normal load, peak load, and a fault-injection run with late, duplicate and malformed events.
- Results go into the test report with the machine specification used.